动态规划

- 动态规划
  - 01 背包
    - cpp
      - 滚动数组写法（从后向前更新）：
    - python
  - 完全背包
      - CPP
      - python
    - 优化
      - cpp
      - python
    - 完全背包与01背包的区别
  - 多重背包
    - cpp
    - python
  - 二进制优化：
    - cpp
    - python
  - 分组背包
    - cpp
    - python
  - 线性dp
    - AcWing 898. 数字三角形
      - 从上而下
      - 从下而上
    - AcWing 895. 最长上升子序列
    - AcWing 896. 最长上升子序列 II
      - 手写二分
      - lower_bound写法
    - AcWing 897. 最长公共子序列
    - **与背包问题的区别**
    - AcWing 902. 最短编辑距离
      - cpp
      - python
    - Acwing899编辑距离
      - cpp
      - python
  - 区间DP
    - AcWing 282. 石子合并
      - cpp
      - python
  - 计数类DP
    - AcWing 900. 整数划分
    - AcWing 338. 计数问题
  - 状态压缩DP
  - AcWing 291. 蒙德里安的梦想
  - AcWing 91. 最短Hamilton路径
  - AcWing 285. 没有上司的舞会

AcWing 901. 滑雪

递归关系

- 状态表示f[i,j]
    - 集合：从前i个物品选且体积小于j
    - 属性：最大值，最小代价，数量
- 状态计算——集合划分

背包问题可以灵活应用于**求恰好容量时的最小/最大操作数。**

01 背包

有 N 件物品和一个容量是 V 的背包。每件物品只能使用一次。

第 i 件物品的体积是 v_i，价值是 w_i。

求解将哪些物品装入背包，可使这些物品的总体积不超过背包容量，且总价值最大。
输出最大价值。

输入格式

第一行两个整数，N，V，用空格隔开，分别表示物品数量和背包容积。

接下来有 N 行，每行两个整数 v_i, w_i，用空格隔开，分别表示第 i 件物品的体积和价值。

输出格式

输出一个整数，表示最大价值。

数据范围

$0 < N, V \leq 1000$
$0 < v_i, w_i \leq 1000$

输入样例

```
4 5
1 2
2 4
3 4
4 5
```

输出样例：

```
8
```

- 每件物品可以用一次

- 不包含第i个物品：f(i - 1, j)
- 包含第i个物品：f(i - 1,j - vi) + wi 空余一个vi的位置，加上第i个物品的权重

cpp

```cpp
#include <algorithm>
#include <iostream>

using namespace std;

const int N = 1010;

int n, m;
int v[N], w[N]; //体积和价值
int f[N][N]; //第一个N表示取第1~n个物品，第二个n表示从1开始到m的容量

int main() {
  cin >> n >> m;
  for (int i = 1; i <= n; i++)
    cin >> v[i] >> w[i];

  for (int i = 1; i <= n; i++)   //从一个物品开始取
    for (int j = 1; j <= m; j++) //逐渐增大背包容量至m
    {

      f[i][j] = f[i - 1][j];    //继承状态，不取第i个物品
      if (j >= v[i])            //若有空间，比较取第i个物品和不取第i个物品的最大值
        f[i][j] = max(f[i][j], f[i - 1][j - v[i]] + w[i]);
    }
  cout << f[n][m] << ' ';
  return 0;
}


4 5
1 2
2 4
3 4
4 5
// 物品是由小到大排列的
v[i] = 1 2 3 4
w[i] = 2 4 4 5
    1 2 3 4 5       j
0   0 0 0 0 0
1   2 2 2 2 2
2   2 4 6 6 6
3   2 4 6 6 8
4   2 4 6 6 8


i

```

滚动数组写法（从后向前更新）：

- 为什么要从后向前更新：因为当你计算 `f[j]` 时，`f[j-v[i]]` 可能已经被当前物品 `i` 更新过了。dp[i]与dp[i-1]有关。这意味着你可能会在同一轮次中多次使用同一个物品，违背了 0/1 背包的要求。

```cpp
#include <algorithm>
#include <iostream>

using namespace std;

const int N = 1010;

int n, m;
int v[N], w[N]; //体积和价值
int f[N]; //采用滚动数组，滚动数组的本质还是减少空间复杂度

int main() {
  cin >> n >> m;
  for (int i = 1; i <= n; i++)
    cin >> v[i] >> w[i];

  for (int i = 1; i <= n; i++)      //从一个物品开始取
    for (int j = m; j >= v[i]; j--) //从后向前更新
    {
      // f[j] = f[j];
      f[j] = max(f[j], f[j - v[i]] + w[i]);
    }
  cout << f[m] << ' ';
  return 0;
}
```

python

```python
n,k = map(int,input().split())

dp = [[0]*(k+10) for _ in range(n+10)]
v = [0] * (n+10)
w = [0] * (n+10)
for i in range(1,n+1):
    a,b = map(int,input().split())
    v[i] = a
    w[i] = b

for i in range(1,n+1):
    for j in range(1,k+1):
        dp[i][j] = dp[i-1][j]    #继承状态，不取第i个物品
        if v[i] <= j:
            dp[i][j] = max(dp[i-1][j],dp[i-1][j-v[i]]+w[i])    #第i个物品只能取一次

print(dp[n][k])
```

滚动数组写法

```python
n,k = map(int,input().split())

dp = [0]*(k+10)
v = [0] * (n+10)
w = [0] * (n+10)
for i in range(1,n+1):
    a,b = map(int,input().split())
    v[i] = a
    w[i] = b

for i in range(1,n+1):
    for j in range(k,v[i]-1,-1):        # 注意range循环左闭右开的特性，应为v[i]-1
        dp[j] = max(dp[j],dp[j-v[i]]+w[i])

print(dp[k])
```

完全背包

有 N 种物品和一个容量是 V 的背包，每种物品都有无限件可用。

第 i 种物品的体积是 v_i，价值是 w_i。

求解将哪些物品装入背包，可使这些物品的总体积不超过背包容量，且总价值最大。
输出最大价值。

输入格式

第一行两个整数，N，V，用空格隔开，分别表示物品种数和背包容积。

接下来有 N 行，每行两个整数 v_i, w_i，用空格隔开，分别表示第 i 种物品的体积和价值。

输出格式

输出一个整数，表示最大价值。

数据范围

$0 < N, V \leq 1000$
$0 < v_i, w_i \leq 1000$

输入样例

```
4 5
1 2
2 4
3 4
4 5
```

输出样例：

```
10
```

- 每个物品有无限个
- 状态表示f[i,j]

    - 集合
    - 属性
- 状态计算——集合划分
- f[i,j] = f[i - 1,j - v[i] * k] + w[i] * k

CPP

```cpp
#include <algorithm>
#include <iostream>

using namespace std;

const int N = 1010;

int n, m;
int v[N], w[N];
int f[N][N];

int main() {
  cin >> n >> m;
  for (int i = 1; i <= n; i++)
    cin >> v[i] >> w[i];

  for (int i = 1; i <= n; i++)
    for (int j = 0; j <= m; j++) //正序更新,依赖于当前f[i]层的数据
      for (int k = 0; k * v[i] <= j; k++)
        f[i][j] = max(f[i][j],
                      f[i - 1][j - k * v[i]] + k * w[i]); //与01背包思想差不多

  cout << f[n][m] << endl;
  return 0;
}

```

python

```python
import sys
sys.setrecursionlimit(100000000)
input = lambda:sys.stdin.readline().strip()

n,m = map(int,input().split())

v = [0]*(n+10)
w = [0]*(n+10)
dp = [[0]*(m+10) for _ in range(n+10)]

for i in range(1,n+1):
    v[i],w[i] = map(int,input().split())

for i in range(1,n+1):
```

```python
    for j in range(1,m+1):
        for k in range(0,j+1):
            if k*v[i]>j:
                break
            dp[i][j] = max(dp[i][j],dp[i-1][j-k*v[i]]+ k*w[i])
            #
print(dp[n][m])

```

优化

优化，由于物品的数量是无限的，因此可以证明：

- f[i,j] = Max(f[i][j], `f[i-1,j-v]+w,f[i-1,j-2v]+2w,f[i-1,j-3v]+3w,...)`等于`f[i,j-v]`
- /f[i,j-v] = Max(， f[i-1,j-v] ,f[i-1,j-2v]+w,f[i-1,j-3v]+2w,...)
- f[i, j] = Max(f[i - 1, j], f[i, j - v] + w)
- 正序更新的目的是让物品能够被多次使用。

cpp

```cpp
#include <iostream>
#include <algorithm>

using namespace std;

const int N = 1010;

int n, m;
int v[N], w[N];
int f[N];

int main()
{
    cin >> n >> m;
    for (int i = 1; i <=  n; i ++ )  cin >> v[i] >> w[i];

    for (int i = 1;i <= n; i ++)
        for (int j = v[i]; j <= m; j ++ )          //正序更新,依赖于当前f[i]层的数据

                    f[j] = max(f[j], f[j - v[i]] + w[i]);
    cout << f[m] << endl;
    return 0;
}

```

python

```python
import sys
sys.setrecursionlimit(100000000)
input = lambda:sys.stdin.readline().strip()

n,m = map(int,input().split())

v = [0]*(n+10)
w = [0]*(n+10)
dp = [0]*(m+10)

for i in range(1,n+1):
    v[i],w[i] = map(int,input().split())

for i in range(1,n+1):
    for j in range(0,m+1):
        if j < v[i]:    #若不加这句，程序能正常运行但答案错误，因为python支持负索引
            continue
        dp[j] = max(dp[j],dp[j-v[i]]+w[i])
print(dp[m])
```

完全背包与01背包的区别

- 01背包：
  - 不优化：f[i, j] = max(f[i - 1, j], `f[i - 1, j- v]` + w)
  - 滚动数组：for (int j = m; j >= v[i]; j--) f[j] = max(f[j], f[j - v[i]] + w[i]);
- 完全背包：
  - 不优化：f[i,j] = max(f[i - 1, j], `f[i, j - v]` + w)
  - 方程化简：for (int j = v[i]; j <= m; j ++) f[j] = max(f[j], f[j - v[i]] + w[i]);
- 优化后循环开始位置不同：01背包是避免覆盖(只能取1个)，完全背包是必须覆盖（尽量取多个）（滑动窗口）

多重背包

- 每个物品有限个
- 在完全背包的基础上限制物品数量

cpp

```cpp
#include <iostream>
#include <utility>
using namespace std;
typedef long long LL;
```

```cpp
const int N = 1e3 +10;
int arr[N];

typedef pair<int, int> PA;
typedef pair<PA, int> PA2;
int n,m;
int v[N],w[N],s[N];
int dp[N][N];


int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    cin >> n >> m;
   for(int i = 1; i<=n; i++)
   {
        cin >> v[i] >> w[i] >> s[i];
   }
   for(int i = 1; i<= n; i++)   //枚举物品
    for(int j = 1; j <= m; j++)   //枚举空间
    {
        dp[i][j] = dp[i-1][j];  // 不选当前物品
        for(int k = 1; k<=s[i] ;k++)    //枚举数量
        {
        //    dp[i][j] = dp[i-1][j];  不能在这使用，每次更新的值会被覆盖掉
                if(j>=v[i]*k)
                {
                    dp[i][j] = max(dp[i][j],dp[i-1][j-k*v[i]]+k*w[i]);
                }
        }
    }

    cout << dp[n][m] << endl;
}

```

python

```python
import sys

input = lambda: sys.stdin.readline().strip()
sys.setrecursionlimit(100000)

n,m = map(int,input().split())

v = [0]*(n+10)
w = [0]*(n+10)
c = [0]*(n+10)
dp = [[0]*(m+10) for _ in range(n+10)]

for i in range(1,n+1):
    v[i],w[i],c[i] = map(int,input().split())
```

```python

for i in range(1,n+1):
    for j in range(1,m+1):
        for k in range(0,c[i]+1):
            if j < v[i]*k:
                continue
            dp[i][j] = max(dp[i][j],dp[i-1][j-v[i]*k] + w[i]*k)

print(dp[n][m])
```

二进制优化：

由于二进制可以表示任意小于2^n的数，因此我们可以将一个物品拆成n个二进制的物品，然后使用滚动数组优化的一维背包解决。

cpp

```cpp
#include <algorithm>
#include <iostream>

using namespace std;

const int N = 25000; // 1000 *log2 (2000)

int n, m;
int v[N], w[N];

int f[N];

int main() {
  cin >> n >> m;
  int cnt = 0;
  for (int i = 1; i <= n; i++) {
    int a, b, s;
    cin >> a >> b >> s;
    int k = 1;
    while (k <= s) // 拆分为多个小背包
    {
      cnt++;
      v[cnt] = a * k; // 体积：原始体积 a 乘以 k
      w[cnt] = b * k; // 价值：原始价值 b 乘以 k
      s -= k;         // 减去已经处理的数量
      k *= 2;         // 乘以 2，进入下一个更大的子背包
    }
    if (s > 0) // 剩余部分作为一个小背包处理
    {
      cnt++;
      v[cnt] = a * s;
      w[cnt] = b * s;
```

```
    }
  }

  //转化为01背包问题
  n = cnt;
  for (int i = 1; i <= n; i++)
    for (int j = m; j >= v[i]; j--)
      f[j] = max(f[j], f[j - v[i]] + w[i]);

  cout << f[m] << endl;

  return 0;
}
```

python

```
import os
import sys
sys.setrecursionlimit(10000)
input = lambda : sys.stdin.readline().strip()

n,m = map(int,input().split())

v = [0]*(n*100)
w = [0]*(n*100)
dp = [0]*(m+10)

cnt = 1
for i in range(n):
    a,b,c = map(int,input().split())
    tmp = 1
    while(tmp <= c):
        c -= tmp
        v[cnt] = a*tmp
        w[cnt] = b*tmp
        cnt+=1
        tmp*=2
    if c:
        v[cnt] = a*c
        w[cnt] = b*c
        cnt+=1
n = cnt-1
for i in range(1,n+1):
    for j in range(m+1,v[i]-1,-1):
        dp[j] = max(dp[j],dp[j-v[i]] + w[i] )


print(dp[m])

```

分组背包

有 N 组物品和一个容量是 V 的背包。

每组物品有若干个，同一组内的物品最多只能选一个。
每件物品的体积是 v_{ij}，价值是 w_{ij}，其中 i 是组号，j 是组内编号。

求解将哪些物品装入背包，可使物品总体积不超过背包容量，且总价值最大。

输出最大价值。

输入格式

第一行有两个整数 N，V，用空格隔开，分别表示物品组数和背包容量。

接下来有 N 组数据：

- 每组数据第一行有一个整数 S_i，表示第 i 个物品组的物品数量；
- 每组数据接下来有 S_i 行，每行有两个整数 v_{ij}, w_{ij}，用空格隔开，分别表示第 i 个物品组的第 j 个物品的体积和价值；

输出格式

输出一个整数，表示最大价值。

数据范围

$0 < N, V \leq 100$
$0 < S_i \leq 100$
$0 < v_{ij}, w_{ij} \leq 100$

输入样例

```
3 5
2
1 2
2 4
1
3 4
1
4 5
```

输出样例：

```
8
```

- 每组只能选一个

类似于01背包问题，我们只需添加组内遍历即可。

cpp

```cpp

#include <iostream>
#include <algorithm>

using namespace std;

const int N = 110;
```

```cpp

int n, m;
int v[N][N], w[N][N], s[N];

int f[N];

int main()
{
    cin >> n >>m;
    for (int i = 1; i <= n; i ++)
    {
        cin >> s[i];
        for (int j = 1; j <= s[i]; j ++)
            cin >> v[i][j] >> w[i][j];
    }

    for (int i = 1; i <= n; i ++ )      //枚举所有组
        for (int j = m; j >= 0; j --)     //枚举所有背包组
            for (int k = 1; k <= s[i]; k ++ )       //枚举组内对象
                if (v[i][k] <= j)
                    f[j] = max(f[j], f[j-v[i][k]] + w[i][k]);

    cout << f[m] <<endl;
return 0;

}
```

python

```python
import sys
sys.setrecursionlimit(100000)

input = lambda : sys.stdin.readline().strip()

n,m = map(int,input().split())
dp = [0] * 120

tot = [0]*105
v = [[0]*(105) for _ in range(n+10)]
w = [[0]*(105) for _ in range(n+10)]
for i in range(1,n+1):
    nn = int(input())
    tot[i] = nn
    for j in range(1,nn+1):
        a,b = map(int,input().split())
        v[i][j] = a
        w[i][j] = b


```

```python
for i in range(1,n+1):
    for j in range(m,-1,-1):
        for k in range(1,tot[i]+1):
            if j < v[i][k]:
                continue
            dp[j] = max(dp[j],dp[j-v[i][k]] + w[i][k])

print(dp[m])


```

线性dp

- 递推顺序是线性的

AcWing 898. 数字三角形

给定一个如下图所示的数字三角形，从顶部出发，在每一结点可以选择移动至其左下方的结点或移动至其右下方的结点，一直走到底层，要求找出一条路径，使路径上的数字的和最大。

```
        7
      3   8
    8   1   0
  2   7   4   4
4   5   2   6   5
```

输入格式

第一行包含整数 n，表示数字三角形的层数。

接下来 n 行，每行包含若干整数，其中第 i 行表示数字三角形第 i 层包含的整数。

输出格式

输出一个整数，表示最大的路径数字和。

数据范围

$1 \le n \le 500$,
$-10000 \le$ 三角形中的整数 ≤ 10000

输入样例：

```
5
7
3 8
8 1 0
2 7 4 4
4 5 2 6 5
```

输出样例：

```
30
```

这题有两种解法：

- 一种是像y总一样从上向下dp，然后对第n行的数据取最大值
- 另一种是从下往上dp，最后直接输出 `dp[1][1]` 即可，由于保证了左下右下两个数必定存在，因此不用初始化和减少了边界可能引发的问题。

从上而下

```cpp
#include <iostream>
#include <algorithm>

using namespace std;

const int N = 510, INF = 1e9;

```

```
int n;
int a[N][N];
int f[N][N];

int main()
{
    scanf("%d", &n);
    for (int i = 1; i <= n; i ++ )
        for (int j = 1; j <= i; j ++ )
            scanf("%d", &a[i][j]);

    for (int i = 0; i <= n; i ++ )          //保证整数为负时max函数的正确性
        for (int j = 0; j <= i + 1; j ++ )
            f[i][j] = -INF;

    f[1][1] = a[1][1];      //第一层dp不成立，特殊处理
    for (int i = 2; i <= n; i ++ )      //注意边界，从第二层开始遍历
        for (int j = 1; j <= i; j ++ )
            f[i][j] = max(f[i - 1][j - 1] + a[i][j], f[i - 1][j] + a[i][j]);

    int res = -INF;
    for (int i = 1; i <= n; i ++ ) res = max(res, f[n][i]);

    printf("%d\n", res);
    return 0;
}
```

从下而上

```
#include<iostream>
#include<cstdio>
#include<cmath>
#include<map>
#include<algorithm>
using namespace std;
int dp[510][510];
int n;
int main()
{
  cin >> n;
  for (int i = 1; i <= n; i++)
  {
    for (int j = 1; j <= i; j++)
    {
      cin >> dp[i][j];
    }
  }
  for (int i = n - 1; i >= 1; i--)
  {
    for (int j = 1; j <= i; j++)
    {
      dp[i][j] = max(dp[i][j] + dp[i + 1][j], dp[i][j] + dp[i + 1][j + 1]);
    }
```

```
  }
  cout << dp[1][1] << endl;
  return 0;

}
```

AcWing 895. 最长上升子序列

给定一个长度为 N 的数列，求数值严格单调递增的子序列的长度最长是多少。

输入格式

第一行包含整数 N。

第二行包含 N 个整数，表示完整序列。

输出格式

输出一个整数，表示最大长度。

数据范围

$1 \leq N \leq 1000$，
$-10^9 \leq$ 数列中的数 $\leq 10^9$

输入样例：

```
7
3 1 2 1 8 5 6
```

输出样例：

```
4
```

我们维持每个位置的最长上升子序列。在枚举到下一元素时，对前面进行遍历寻求最长序列。

```
#include <iostream>

using namespace std;

const int N = 1e5;
int a[N], dp[N];
int main()
{
    int n;
    cin >> n;
    for (int i = 0; i < n; i++)
    {
        cin >> a[i];
    }
    for(int i = 0; i < n; i++)
    {
```

```
        dp[i] = 1;        //初始化为最短上升子序列，最小长度为1
        for(int j = 0; j < i; j++)
        {
             if(a[j] < a[i])      //构成上升子序列，更新
                 dp[i] = max(dp[i], dp[j] + 1);
        }
    }
    int maxnum = -1e9;       //考虑子序列为负的情况
    for(int i = 0; i < n; i++)
    {
        maxnum = max(maxnum, dp[i]);
    }
    cout<< maxnum << endl;
}
```

AcWing 896. 最长上升子序列 II

给定一个长度为 N 的数列，求数值严格单调递增的子序列的长度最长是多少。

输入格式

第一行包含整数 N。

第二行包含 N 个整数，表示完整序列。

输出格式

输出一个整数，表示最大长度。

数据范围

$1 \leq N \leq 100000$,
$-10^9 \leq$ 数列中的数 $\leq 10^9$

输入样例：

```
7
3 1 2 1 8 5 6
```

输出样例：

```
4
```

利用贪心思想（维护最小末尾）和二分查找优化。

手写二分

```
#include <iostream>
#include <algorithm>

using namespace std;
const int N = 100010;
```

```
int n;
int a[N];
int q[N];

int main()
{
    cin >> n;

    for (int i = 0; i < n; i ++ )
        cin >> a[i];
    int len = 0;
    for (int i = 0; i < n; i ++ )
    {
       int l = 0, r = len;
       while( l < r)
       {
         int mid = (l + r + 1) >> 1;
         if( q[mid] < a[i] )    //保证是最大小于a[i]的位置，并覆盖q[r+1]
            l = mid;
         else
            r = mid - 1;
       }
       len = max(len, r + 1);
       q[r + 1] = a[i];
    }

    printf("%d\n", len);
    return 0;
}
```

lower_bound写法

```
#include <algorithm>
#include <iostream>
using namespace std;

const int N = 1e5 + 10;
int arr[N];
int brr[N];  // brr[i] 存储长度为 i 的 LIS 的最小末尾值

int main() {
    ios::sync_with_stdio(0);
    cin.tie(0);
    int n;
    cin >> n;
    for (int i = 1; i <= n; i++) {
        cin >> arr[i];
    }

    int len = 0;
    for (int i = 1; i <= n; i++) {
        // 在 brr[1..len] 中找第一个 ≥ arr[i] 的位置
        int pos = lower_bound(brr + 1, brr + len + 1, arr[i]) - brr;
```

```
        if (pos > len) {
            len++;
            brr[len] = arr[i];  // 扩展 LIS 长度
        } else {
            brr[pos] = arr[i];  // 更新更小的末尾值
        }
    }

    cout << len << endl;
    return 0;
}
```

AcWing 897. 最长公共子序列

给定两个长度分别为 N 和 M 的字符串 A 和 B，求既是 A 的子序列又是 B 的子序列的字符串长度最长是多少。

输入格式

第一行包含两个整数 N 和 M。

第二行包含一个长度为 N 的字符串，表示字符串 A。

第三行包含一个长度为 M 的字符串，表示字符串 B。

字符串均由小写字母构成。

输出格式

输出一个整数，表示最大长度。

数据范围

$1 \leq N, M \leq 1000$

输入样例：

```
4 5
acbd
abedc
```

输出样例：

```
3
```

如果字母相同，则可以取`f[i][j] = f[i - 1][j - 1] + 1;`，若不同，由于我们前面序列是确定的（即最长公共子序列），可以确保第i，j时，最长子序列的长度不变,即`f[i][j] = max(f[i - 1][j], f[i][j - 1]);`，

```
#include <iostream>
using namespace std;
const int N = 1010;
int n, m;
```

```
char a[N], b[N];
int f[N][N];
int main() {
  cin >> n >> m >> a + 1 >> b + 1;  //加号优先级大于>>
  for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++) {
        if(a[i] == b[j])
            f[i][j] = f[i - 1][j - 1] + 1;  // 继承
        else
            f[i][j] = max(f[i - 1][j], f[i][j - 1]);
    }
    cout << f[n][m] << endl;
}
```

从dp数组上分析

```
  a c b d
a 1 0 0 0
b 0 0 1 0
e 0 0 0 0
d 0 0 0 1
c 0 1 0 0              向右下走（不能往左下走）到达f[n][m]能达到的最大值
```

与背包问题的区别

- **背包问题**：
  `f[i][j] = max(f[i-1][j], f[i-1][j-v[i]] + w[i])`
  - `f[i-1][j]`：明确表示不取第 `i` 个物品。
  - `f[i-1][j-v[i]] + w[i]`：明确表示取第 `i` 个物品。
- **LCS 问题**：
  `f[i][j] = max(f[i-1][j], f[i][j-1])`
  - `f[i-1][j]` 和 `f[i][j-1]` 是 **两种可能的匹配路径**，而非“取/不取”物品。
  - 目标是 **最大化公共序列长度**，而非“最大化价值”。

AcWing 902. 最短编辑距离

给定两个字符串 A 和 B，现在要将 A 经过若干操作变为 B，可进行的操作有：

1. 删除–将字符串 A 中的某个字符删除。
2. 插入–在字符串 A 的某个位置插入某个字符。
3. 替换–将字符串 A 中的某个字符替换为另一个字符。

现在请你求出，将 A 变为 B 至少需要进行多少次操作。

输入格式

第一行包含整数 n，表示字符串 A 的长度。

第二行包含一个长度为 n 的字符串 A。

第三行包含整数 m，表示字符串 B 的长度。

第四行包含一个长度为 m 的字符串 B。

字符串中均只包含大小写字母。

输出格式

输出一个整数，表示最少操作次数。

数据范围

$1 \le n, m \le 1000$

输入样例：

```
10
AGTCTGACGC
11
AGTAAGTAGGC
```

输出样例：

```
4
```

[AcWing 902. 最短编辑距离 - AcWing]

```
1)删除操作：把a[i]删掉之后a[1~i]和b[1~j]匹配
            所以之前要先做到a[1~(i-1)]和b[1~j]匹配
            f[i-1][j] + 1
2)插入操作：插入之后a[i]与b[j]完全匹配，所以插入的就是b[j]
            那填之前a[1~i]和b[1~(j-1)]匹配
            f[i][j-1] + 1
3)替换操作：把a[i]改成b[j]之后想要a[1~i]与b[1~j]匹配
            那么修改这一位之前，a[1~(i-1)]应该与b[1~(j-1)]匹配
            f[i-1][j-1] + 1
            但是如果本来a[i]与b[j]这一位上就相等，那么不用改，即
            f[i-1][j-1] + 0
```

cpp

```cpp
#include <algorithm>
#include <iostream>

using namespace std;

const int N = 1010;

int n, m;
char a[N], b[N];
int f[N][N];

int main() {
  scanf("%d%s", &n, a + 1);
  scanf("%d%s", &m, b + 1);
  for (int i = 0; i <= n; i++)      //将前i个字母转为空串
    f[i][0] = i;
  for (int i = 0; i <= m; i++)       //将前i个字母转为空串
    f[0][i] = i;
  for (int i = 1; i <= n; i++)
    for (int j = 1; j <= m; j++) {
      f[i][j] = min(f[i - 1][j] + 1, f[i][j - 1] + 1);
      if (a[i] == b[j])
        f[i][j] = min(f[i][j], f[i - 1][j - 1]);
      else
        f[i][j] = min(f[i][j], f[i - 1][j - 1] + 1);
    }
  cout << f[n][m] << endl;
}

```

python

```python

n = int(input())
a = " " + input().strip()
m = int(input())
b = " " + input().strip()

f = [[0] * (m + 1) for _ in range(n + 1)]
for i in range(n + 1):
    f[i][0] = i     #令前i个字符串变成0空串，需要i步
for j in range(m + 1):
    f[0][j] = j      #令空串变成前i个字符串，需要i步

for i in range(1, n + 1):
    for j in range(1, m + 1):
        # 情况1：删除 a[i]（操作数 +1）
```

```python
        # 情况2：插入 b[j]（操作数 +1）
        f[i][j] = min(f[i - 1][j] + 1, f[i][j - 1] + 1)
        # 情况3：替换或匹配
        if a[i] == b[j]:
            f[i][j] = min(f[i][j], f[i - 1][j - 1])  # 字符相同，无需操作
        else:
            f[i][j] = min(f[i][j], f[i - 1][j - 1] + 1)  # 替换操作


print(f[n][m])
```

Acwing899编辑距离

给定 n 个长度不超过 10 的字符串以及 m 次询问，每次询问给出一个字符串和一个操作次数上限。

对于每次询问，请你求出给定的 n 个字符串中有多少个字符串可以在上限操作次数内经过操作变成询问给出的字符串。

每个对字符串进行的单个字符的插入、删除或替换算作一次操作。

输入格式

第一行包含两个整数 n 和 m。

接下来 n 行，每行包含一个字符串，表示给定的字符串。

再接下来 m 行，每行包含一个字符串和一个整数，表示一次询问。

字符串中只包含小写字母，且长度均不超过 10。

输出格式

输出共 m 行，每行输出一个整数作为结果，表示一次询问中满足条件的字符串个数。

数据范围

$1 \le n, m \le 1000$,

输入样例：

```
3 2
abc
acd
bcd
ab 1
acbd 2
```

输出样例：

```
1
3
```

cpp

这里采用编辑最短距离的思路，但是这题要注意`strlen()`函数，由于我们输入的位置是arr+1，因此字符串的首地址不再是pos，而是pos+1。

```cpp
#include <iostream>
#include <string.h>

using namespace std;

const int N = 15, M = 1010;
int f[N][N], x;
char str[M][N], a[N];

int main() {
  int n;
  cin >> n;
  int m;
  cin >> m;
  for (int i = 0; i < n; i++)
    cin >> str[i] + 1;

  while (m--) {
    int x;
    cin >> a + 1 >> x;
    int cnt = 0;

    for (int pos = 0; pos < n; pos++) {
      int lenpos = strlen(str[pos] + 1);
      int lena = strlen(a + 1);
      for (int i = 0; i <= lenpos; i++)
        f[i][0] = i;
      for (int i = 1; i <= lena; i++)
        f[0][i] = i;
      for (int i = 1; i <= lenpos; i++)
        for (int j = 1; j <= lena; j++) {
          f[i][j] = min(f[i - 1][j] + 1, f[i][j - 1] + 1);
          if (str[pos][i] == a[j])
            f[i][j] = min(f[i - 1][j - 1], f[i][j]);
          else
            f[i][j] = min(f[i - 1][j - 1] + 1, f[i][j]);
        }
      if (x >= f[lenpos][lena])
        cnt++;
    }
    cout << cnt << endl;
  }
}
```

python

```python
n,m = map(int,input().split())

stra = []
for i in range(n):
    stra.append(" "+input())

strb = []
cnt = []
for j in range(m):
    str,tmp  = input().split()
    strb.append(" "+str)
    cnt.append(int(tmp))

dp = [[0]*13 for _ in range(13)]    #要加入0才是整型数组

def judge(n1):
    ss = strb[n1]
    res = 0

    for i in range(n):
        sss = stra[i]
        for j in range(12):
            #print(1)
            dp[0][j] = j
        for j in range(12):
            dp[j][0] = j

        for j in range(1,len(sss)):
            for k in range(1,len(ss)):

                dp[j][k] = min(dp[j][k-1]+1,dp[j-1][k]+1)
                if sss[j] == ss[k]:
                    dp[j][k] = min(dp[j][k],dp[j-1][k-1])
                else:
                    dp[j][k] = min(dp[j][k],dp[j-1][k-1]+1)
        if dp[len(sss)-1][len(ss)-1] <= cnt[n1]:
            res += 1


    print(res)


for i in range(m):
    judge(i)


```

区间DP

AcWing 282. 石子合并

设有 N 堆石子排成一排，其编号为 $1, 2, 3, \ldots, N$。

每堆石子有一定的质量，可以用一个整数来描述，现在要将这 N 堆石子合并成为一堆。

每次只能合并相邻的两堆，合并的代价为这两堆石子的质量之和，合并后与这两堆石子相邻的石子将和新堆相邻，合并时由于选择的顺序不同，合并的总代价也不相同。

例如有 4 堆石子分别为 `1 3 5 2`， 我们可以先合并 1、2 堆，代价为 4，得到 `4 5 2`， 又合并 1、2 堆，代价为 9，得到 `9 2` ，再合并得到 11，总代价为 $4 + 9 + 11 = 24$；

如果第二步是先合并 2、3 堆，则代价为 7，得到 `4 7`，最后一次合并代价为 11，总代价为 $4 + 7 + 11 = 22$。

问题是：找出一种合理的方法，使总的代价最小，输出最小代价。

输入格式

第一行一个数 N 表示石子的堆数 N。

第二行 N 个数，表示每堆石子的质量(均不超过 1000)。

输出格式

输出一个整数，表示最小代价。

数据范围

$1 \le N \le 300$

输入样例：

```
4
1 3 5 2
```

输出样例：

```
22
```

这里是从每次合并二堆开始枚举，直到最后一次合并

$f[i][j]$ 表示将 i 到 j这一段石子合并成一堆的方案的集合

1. 从最小区间2开始枚举，记左边界为l，右边界为r
2. 取k为区间分割点包含于(l,r)
3. 将区间 `[l, r]` 分割为 `[l, k]` 和 `[k + 1, r]`，分别计算这两部分的最小合并代价，加上合并这两部分的代价（即区间 `[l, r]` 的石子总数 `s[r] - s[l - 1]` ）。
4.

cpp

```cpp
#include <iostream>


using namespace std;

const int N = 310;
int n;
int s[N];
int f[N][N];


int main(){
    cin >> n;
    for(int i = 1; i <= n; i ++) cin >> s[i];
    for(int i = 1; i<= n; i ++)
        s[i] += s[i - 1];       //前缀和
    for(int len = 2; len<=n; len++)     //每次合并长度
        for(int i = 1; i+ len - 1 <= n; i++)
        {
            int l = i;
            int r = i + len - 1;        //每次合并多少石子
            f[l][r] = 1e8;      //初始化为无穷大
            for(int k = l; k <= r; k++)
            {
                f[l][r] = min(f[l][r], f[l][k] +f[k+1][r] + s[r] - s[l - 1])
            ;}

        }
        cout << f[1][n] << endl;
        return 0;
}
```

python

```python
dp = [[0]*310 for _ in range(310)]

n = int(input())

arr = [0] + list(map(int, input().split()))
brr = [0]*310

for i in range(1,n+1):  #前缀和
    brr[i] = arr[i] + brr[i-1]

for len in range(2,n+1):      #枚举合并大小
    for j in range(1,n-len+1+1):    #枚举合并的起始位置
        l = j
        r = j+len-1
        dp[l][r] = 0x3f3f3f3f
        for k in range(l,r):
```

```
            dp[l][r] = min(dp[l][r],dp[l][k]+dp[k+1][r] + brr[r] - brr[l-1])

print(dp[1][n])
```

计数类DP

AcWing 900. 整数划分

一个正整数 n 可以表示成若干个正整数之和，形如：$n = n_1 + n_2 + \ldots + n_k$，其中 $n_1 \geq n_2 \geq \ldots \geq n_k, k \geq 1$。

我们将这样的一种表示称为正整数 n 的一种划分。

现在给定一个正整数 n，请你求出 n 共有多少种不同的划分方法。

输入格式

共一行，包含一个整数 n。

输出格式

共一行，包含一个整数，表示总划分数量。

由于答案可能很大，输出结果请对 $10^9 + 7$ 取模。

数据范围

$1 \leq n \leq 1000$

输入样例:

```
5
```

输出样例:

```
7
```

这里采用完全背包的方式，从1开始到n，由于我们一开始是用1枚举的，因此可以保证背包是满的。

- `f[i][j]` 表示用前 `i` 个正整数（即 `1, 2, ..., i`）组成 `j` 的方案数。
- 求最大值时，当都不选时，价值显然是 0
- 而求方案数时，当都不选时，方案数是 1（即前 i 个物品都不选的情况也是一种方案）

```
#include <iostream>


using namespace std;

const int N = 1e3 + 7, mod = 1e9 + 7;

int f[N][N];
```

```

int main() {
    int n;
    cin >> n;

    for (int i = 0; i <= n; i ++) {
        f[i][0] = 1; // 容量为0时，前 i 个物品全不选也是一种方案
    }

for (int i = 1; i <= n; i ++) {        // 枚举正整数 1 到 n
    for (int j = 0; j <= n; j ++) {    // 枚举目标数 0 到 n
        f[i][j] = f[i - 1][j] % mod;   // 不选当前数 i 的情况
        if (j >= i) f[i][j] = (f[i - 1][j] + f[i][j - i]) % mod; // 选当前数 i 的情况
    }
}

    cout << f[n][n] << endl;
}
```

AcWing 338. 计数问题

给定两个整数 a 和 b，求 a 和 b 之间的所有数字中 $0 \sim 9$ 的出现次数。

例如，$a = 1024$，$b = 1032$，则 a 和 b 之间共有 9 个数如下：

`1024 1025 1026 1027 1028 1029 1030 1031 1032`

其中 `0` 出现 10 次，`1` 出现 10 次，`2` 出现 7 次，`3` 出现 3 次等等…

输入格式

输入包含多组测试数据。

每组测试数据占一行，包含两个整数 a 和 b。

当读入一行为 `0 0` 时，表示输入终止，且该行不作处理。

输出格式

每组数据输出一个结果，每个结果占一行。

每个结果包含十个用空格隔开的数字，第一个数字表示 `0` 出现的次数，第二个数字表示 `1` 出现的次数，以此类推。

数据范围

$0 < a, b < 100000000$

输入样例：

```
1 10
44 497
346 542
1199 1748
1496 1403
1004 503
1714 190
1317 854
1976 494
1001 1960
0 0
```

输出样例：

```
1 2 1 1 1 1 1 1 1 1
85 185 185 185 190 96 96 96 95 93
40 40 40 93 136 82 40 40 40 40
115 666 215 215 214 205 205 154 105 106
16 113 19 20 114 20 20 19 19 16
107 105 100 101 101 197 200 200 200 200
413 1133 503 503 503 502 502 417 402 412
196 512 186 104 87 93 97 97 142 196
398 1375 398 398 405 499 499 495 488 471
294 1256 296 296 296 296 287 286 286 247
```

这题我感觉像是模拟+前缀和，分析xxxjyyy的各种情况

[AcWing 338. 计数问题<超短写法> - AcWing]

```cpp

#include <algorithm>
#include <cmath>
#include <iostream>
#include <utility>

using namespace std;

int get(int x) {
  int res = 0;
  while (x)
    res++, x /= 10;
  return res;
}

int count(int n, int i) {
  int res = 0, dgt = get(n);
  for (int j = 1; j <= dgt; ++j) {
    // p为当前遍历位次数大小
    // l为第j位的左边的数
    // r为右边的数
    // dj为第j位上的数
    int p = pow(10, dgt - j);
    int l = n/p/10;
    int r = n % p;
    int dj = n / p % 10;

    if(i)   res+=  l*p;
    else    res+= (l - 1) * p;  //去除00...0的情况

    if (i == dj)     res += r + 1;      //等于需小于等于r
    if (i < dj)             res += p;   //小于可取全部值
    //大于则无

  }
  return res;
}

int main() {
  int a, b;
  while (cin >> a >> b, a) { // 输入处理，直到输入为0 0停止
    if (a > b)
      swap(a, b);
    for (int i = 0; i <= 9; ++i)
      cout << count(b, i) - count(a - 1, i) << ' ';
    //输出各位数字的个数
    //利用前缀和思想：[l,r]的和=s[r] - s[l - 1]
    cout << endl;
  }

  return 0;
}

```

状态压缩DP

AcWing 291. 蒙德里安的梦想

摆放的小方格方案数 等价于 横着摆放的小方格方案数

当我们确定唯一的横放小方格数量时，竖着放的小方格位置就已经确定了

- 连续空着的列需要是偶数个
- 该位置未被其他横放小方块占据

//TODO

AcWing 91. 最短Hamilton路径

[AcWing 91. 最短Hamilton路径(超详解) - AcWing]

给定一张 n 个点的带权无向图，点从 $0 \sim n-1$ 标号，求起点 0 到终点 $n-1$ 的最短 Hamilton 路径。

Hamilton 路径的定义是从 0 到 $n-1$ 不重不漏地经过每个点恰好一次。

输入格式

第一行输入整数 n。

接下来 n 行每行 n 个整数，其中第 i 行第 j 个整数表示点 i 到 j 的距离（记为 $a[i,j]$）。

对于任意的 x,y,z，数据保证 $a[x,x]=0$，$a[x,y]=a[y,x]$ 并且 $a[x,y]+a[y,z] \geq a[x,z]$。

输出格式

输出一个整数，表示最短 Hamilton 路径的长度。

数据范围

$1 \leq n \leq 20$
$0 \leq a[i,j] \leq 10^7$

输入样例：

```
5
0 2 4 5 1
2 0 6 5 3
4 6 0 8 3
5 5 8 0 5
1 3 3 5 0
```

输出样例：

```
18
```

`f[i][j]=min(f[i][j],f[i-(1<<j)][k]+w[k][j])`

```cpp
#include <algorithm>
#include <cstring>
#include <iostream>

using namespace std;

const int N = 20, M = 1 << N;

int f[M][N], w[N][N]; // w表示的是无权图

int main() {
  int n;
  cin >> n;

  for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
      cin >> w[i][j];
  memset(f, 0x3f, sizeof(f));
  f[1][0] = 0;      //000.。。1     的状态，初始化为0
  for (int i = 0; i < (1 << n); i++)          //枚举所有状态
    for (int j = 0; j < n; j++)     //枚举所有点
```

```
      if (i >> j & 1)        //判断j位是否为1，为1
        for (int k = 0; k < n; k++)      //k表示走到j这个点之前,以k为终点的最短距离
          if (i - (1 << j) >> k & 1)
            f[i][j] = min(f[i][j], f[i - (1 << j)][k] + w[k][j]);

  cout << f[(1 << n) - 1][n - 1] << endl;
  // 11111111的情况
}
```

AcWing 285. 没有上司的舞会

Ural 大学有 N 名职员，编号为 $1 \sim N$。

他们的关系就像一棵以校长为根的树，父节点就是子节点的直接上司。

每个职员有一个快乐指数，用整数 H_i 给出，其中 $1 \leq i \leq N$。

现在要召开一场周年庆宴会，不过，没有职员愿意和直接上司一起参会。

在满足这个条件的前提下，主办方希望邀请一部分职员参会，使得所有参会职员的快乐指数总和最大，求这个最大值。

输入格式

第一行一个整数 N。

接下来 N 行，第 i 行表示 i 号职员的快乐指数 H_i。

接下来 $N-1$ 行，每行输入一对整数 L, K，表示 K 是 L 的直接上司。（注意一下，后一个数是前一个数的**父节点**，不要搞反）。

输出格式

输出最大的快乐指数。

数据范围

$1 \leq N \leq 6000$,
$-128 \leq H_i \leq 127$

输入样例：

```
7
1
1
1
1
1
1
1
1 3
2 3
6 4
7 4
4 5
3 5
```

输出样例：

```
5
```

我们可以设置一个节点的状态为选中或未选中

```
#include <cstring>
#include <iostream>
#include <algorithm>

```

```
using namespace std;

const int N = 6010;

int n;
int h[N], e[N], ne[N], idx;
int happy[N];
int f[N][2];
bool has_fa[N]; //判断当前节点是否有父节点

void add(int a, int b)
{
    e[idx] = b;
    ne[idx] = h[a];
    h[a] = idx ++;
}

void dfs(int u){
    f[u][1] = happy[u]; //节点u的快乐值，不选节点u为0，全局变量设置
    for(int i = h[u]; ~i; i = ne[i])
    {
        int j = e[i];
        dfs(j);
        f[u][1] += f[j][0];     //选父节点则不选子节点
        f[u][0] += max(f[j][0], f[j][1]);   //不选父节点，则选择子节点的最大快乐值
    }
}


int main()
{
    scanf("%d", &n);

    for (int i = 1; i <= n; i ++ ) scanf("%d", &happy[i]);

    memset(h, -1, sizeof h);
    for (int i = 0; i < n - 1; i ++ )
    {
        int a, b;
        scanf("%d%d", &a, &b);
        add(b, a);      //反向建边，从根节点到叶子节点
        has_fa[a] = true;
    }
    int root = 1;
    while(has_fa[root]) //找到根节点
        root ++;
    dfs(root);
    printf("%d\n", max(f[root][0], f[root][1]))

}

```

AcWing 901. 滑雪

给定一个 R 行 C 列的矩阵，表示一个矩形网格滑雪场。

矩阵中第 i 行第 j 列的点表示滑雪场的第 i 行第 j 列区域的高度。

一个人从滑雪场中的某个区域内出发，每次可以向上下左右任意一个方向滑动一个单位距离。

当然，一个人能够滑动到某相邻区域的前提是该区域的高度低于自己目前所在区域的高度。

下面给出一个矩阵作为例子：

```
 1  2  3  4 5

16 17 18 19 6

15 24 25 20 7

14 23 22 21 8

13 12 11 10 9
```

在给定矩阵中，一条可行的滑行轨迹为 $24-17-2-1$。

在给定矩阵中，最长的滑行轨迹为 $25-24-23-...-3-2-1$，沿途共经过 25 个区域。

现在给定你一个二维矩阵表示滑雪场各区域的高度，请你找出在该滑雪场中能够完成的最长滑雪轨迹，并输出其长度(可经过最大区域数)。

输入格式

第一行包含两个整数 R 和 C。

接下来 R 行，每行包含 C 个整数，表示完整的二维矩阵。

输出格式

输出一个整数，表示可完成的最长滑雪长度。

数据范围

$1 \le R, C \le 300$,
$0 \le$ 矩阵中整数 ≤ 10000

输入样例：

```
5 5
1 2 3 4 5
16 17 18 19 6
15 24 25 20 7
14 23 22 21 8
13 12 11 10 9
```

输出样例：

```
25
```

`f[i][j]`表示从该点出发可以达到的最大位置

```cpp
#include <iostream>
#include <algorithm>
#include <cstring>
using namespace std;
const int N = 310;
int n,m;
int f[N][N];
int h[N][N];
int dx[4] = {-1,0,1,0};
int dy[4] = {0,1,0,-1};


int dp(int x, int y){
    int &v = f[x][y];
      if(v != -1) return v;
        v = 1; //注意v要先赋值为1哦~
    for(int i = 0;i < 4;i ++){ //四个方向
        int xx = x + dx[i];
        int yy = y + dy[i];
        if(xx >= 1 && xx <= n && yy >= 1 && yy <= m && h[x][y] > h[xx][yy]){ //判断这点是否能走
            v = max(v,dp(xx,yy) + 1); //更新
        }
    }
    return v;
}

int main(){
    cin>>n>>m;
    for(int i = 1;i <= n;i ++){
        for(int j = 1;j <= m;j ++){
            cin>>h[i][j];
        }
    }
    memset(f,-1,sizeof f);  //初始化
    int res = 0;
    for(int i = 1;i <= n;i ++){
        for(int j = 1;j <= m;j ++){

            res = max(res,dp(i,j));
        }
    }
    cout<<res<<endl;
    return 0;
}

```